03 — Backtest 001.1: cruzamento de médias + walk-forward + machine learning

LHS Quant Lab · notebook didático

De onde viemos

No **Backtest 001** (notebook `02_primeiro_backtest`) testamos a regra mais simples de *trend following* no EURUSD H4: **SMA 20 acima da SMA 50 → comprado, abaixo → vendido**. Resultado: **reprovado** (−20% líquido, Sharpe −0,12, percentil 41 contra sinais aleatórios).

Aquele teste tinha **parâmetros fixos** e **nenhum aprendizado de máquina**. Esta versão, a **001.1**, mantém a mesma regra base e acrescenta duas ferramentas para ver se elas salvam a estratégia:

Ferramenta	Pergunta que responde
Otimização walk-forward	Se eu reescolher as médias todo ano, olhando só o passado, a estratégia melhora *fora da amostra*?
Filtro de machine learning (meta-labeling)	Um modelo consegue prever **quais cruzamentos** vão dar lucro e pular os ruins?

> **Por que "1.1" e não "002"?** Porque a regra base é a mesma do 001. Só mudam as camadas por cima dela.

O cuidado número 1 deste notebook: não enganar a si mesmo

Otimização e machine learning são as duas formas **mais fáceis** de fabricar um backtest bonito que quebra na conta real (*overfitting*). Por isso, aqui, **toda** decisão é tomada só com dados que já existiam naquele momento:

- a otimização escolhe as médias na janela de **treino** e é avaliada na janela **seguinte**, que ela nunca viu;
- o modelo de ML é **retreinado todo ano** só com operações **já encerradas** antes do ano testado;
- o resultado do ML é comparado com **filtros aleatórios** que pulam a mesma quantidade de operações.

Roteiro

1. Configuração
2. Dados e funções do motor de backtest

3. Conferência: reproduzir o Backtest 001
4. Otimização walk-forward
5. Machine learning: o que é meta-labeling
6. Features (as "pistas" do modelo)
7. Treino walk-forward dos modelos
8. O filtro funcionou? Qualidade das previsões
9. Filtro de ML × filtros aleatórios
10. Comparação final (fora da amostra, 2014 → hoje)
11. Salvar os resultados
12. Conclusões e exercícios

1. Configuração

Tudo o que pode mudar fica aqui.

- **Regra base:** `MEDIA_RAPIDA` × `MEDIA_LENTA` = 20 × 50, igual ao 001.
- **Custos:** spread real de cada barra + `SLIPPAGE_PONTOS` por lado + comissão.
- **Walk-forward:** janela de treino de `ANOS_TREINO` anos, janela de teste de `ANOS_TESTE` ano, andando de ano em ano. O primeiro ano testado é `INICIO_OOS` (2014), porque precisamos de 4 anos de treino a partir de 2010.
- `GRADE_RAPIDAS` / `GRADE_LENTAS`: as 19 combinações candidatas (as mesmas do mapa de calor do 001).
- `MIN_TRADES_TREINO`: uma combinação só pode ser escolhida se tiver operado pelo menos isso no treino (evita escolher algo que "acertou" 3 operações por sorte).
- **ML:** `PROB_MINIMA` é a probabilidade de acerto que o modelo precisa prever para a operação ser aceita. `MIN_TRADES_ML` é o mínimo de operações encerradas para treinar o modelo.
- `SEED`: fixa toda a aleatoriedade (modelos e filtros aleatórios).

2. Dados e funções do motor de backtest

Mesmos dados do 001 (XM via MT5, banco Parquet). Em vez de repetir o código passo a passo, juntamos o motor em **funções pequenas**, porque vamos rodar dezenas de backtests:

Função	O que faz
`posicao_sma(r, l)`	sinal na barra **fechada** (+1/−1) e posição com `shift(1)`: decide em *t*, carrega em *t+1*
`retornos(pos)`	retorno por barra = posição × retorno do mercado − custo da operação completa na entrada
`lista_trades(pos, ret)`	agrupa as barras em operações (entrada, saída, direção, resultado líquido)

Função	O que faz
`metricas(ret, trades)`	retorno, CAGR, nº de trades, win rate, payoff, expectativa, fator de lucro, drawdown, Sharpe, Sortino

Um detalhe importante: `retornos()` recebe **qualquer** série de posições. Assim, tanto a posição montada pelo walk-forward quanto a posição filtrada pelo ML pagam os custos **exatamente da mesma forma**.

```
EURUSD H4: 26,119 barras de 2010-01-04 a 2026-10-16
Duplicadas: 0 | fechamentos vazios: 0 | barras/ano: 1,557
```

3. Conferência: reproduzir o Backtest 001

Antes de acrescentar qualquer coisa, o motor novo precisa dar **o mesmo número** do motor antigo. Se a regra 20×50 no período completo não der ≈ −20,2% de retorno e 586 operações, há um erro no código (ou os dados foram atualizados — aí a diferença deve ser pequena).

```
Reprodução do 001 -> retorno -20.2% | trades 586 | Sharpe -0.12
Esperado (17/10/2026) -> retorno -20.2% | trades 586 | Sharpe -0.12
```

4. Otimização walk-forward

O problema da otimização comum

No 001, o mapa de calor mostrou que a melhor combinação **in-sample** foi 20×200. Escolher a melhor célula olhando o passado inteiro e depois dizer "essa estratégia dá X%" é **trapaça involuntária**: com 19 tentativas, alguma sempre vai parecer boa por sorte.

Como o walk-forward resolve

Simulamos o que um trader faria de verdade, ano a ano:

```
treino 2010-2013 → escolhe as médias → opera 2014 com elas
treino 2011-2014 → escolhe de novo   → opera 2015
...                                    ...
treino 2022-2025 → escolhe de novo   → opera 2026
```

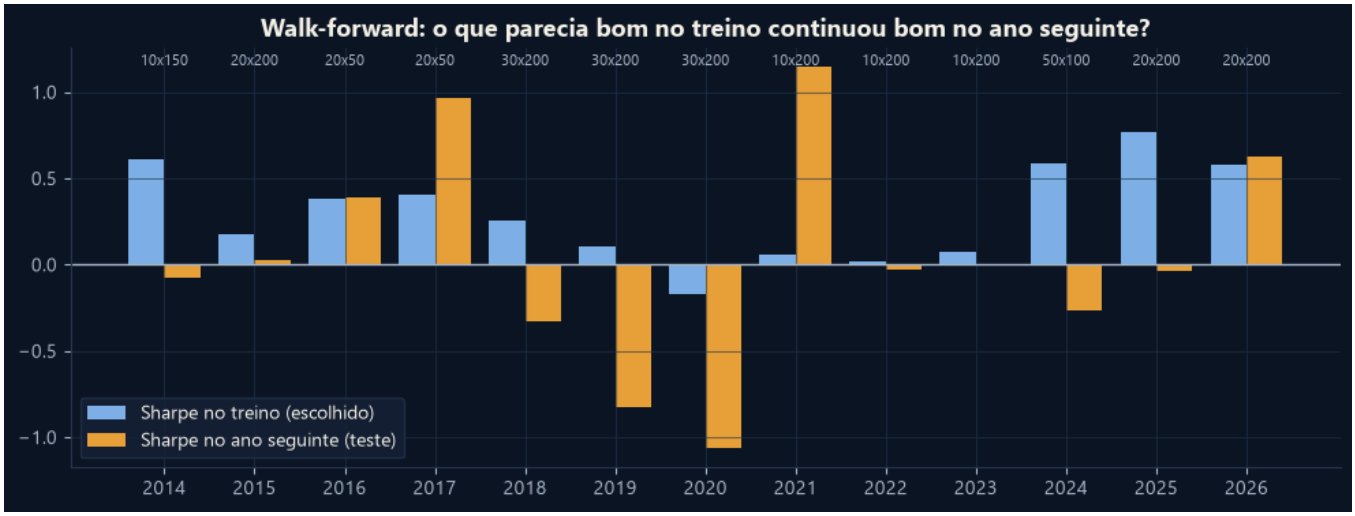
1. Em cada janela de **treino** (4 anos), calculamos o Sharpe líquido das 19 combinações.
2. Escolhemos a de maior Sharpe (com pelo menos `MIN_TRADES_TREINO` operações).
3. Usamos essa combinação **só no ano seguinte**, que o treino nunca viu.
4. Juntamos os 13 anos de teste numa única curva: esse é o resultado **honesto** da otimização.

Ao trocar de combinação na virada do ano, a posição pode mudar. A função `retornos()` cobra o custo dessa troca normalmente.

Eficiência walk-forward (WFE) = Sharpe médio no teste ÷ Sharpe médio no treino. Perto de 1 (ou mais) = o que foi aprendido se mantém. Perto de 0 ou negativo = a otimização só decorou o passado.

```
Combinações escolhidas: {'20x200': 3, '30x200': 3, '10x200': 3, '20x50': 2, '10x150': 1, '50x
100': 1}
Sharpe médio treino 0.30 | teste 0.04 | WFE 0.14
Anos em que a escolha otimizada bateu a 20x50 fixa: 9 de 13
```

ano_teste	escolha	sharpe_treino	sharpe_teste	sharpe_teste_20x50
2014	10x150	0.6100	-0.0800	-0.8700
2015	20x200	0.1800	0.0200	0.1800
2016	20x50	0.3800	0.3900	0.3900
2017	20x50	0.4100	0.9700	0.9700
2018	30x200	0.2500	-0.3300	-1.7400
2019	30x200	0.1000	-0.8300	-1.7100
2020	30x200	-0.1700	-1.0600	0.0300
2021	10x200	0.0600	1.1500	-0.4400
2022	10x200	0.0200	-0.0300	-0.2300
2023	10x200	0.0800	0.0000	-0.5300
2024	50x100	0.5900	-0.2600	-0.6300
2025	20x200	0.7700	-0.0300	-0.1400
2026	20x200	0.5800	0.6300	-0.1900



Como ler: se as barras âmbar (teste) acompanham as azuis (treino), a otimização captura algo real. Se as azuis são sempre positivas e as âmbar oscilam em torno de zero, o "melhor parâmetro" de cada janela era só o que **teve mais sorte** naquela janela. Repare também se a combinação escolhida muda muito de

um ano para o outro: parâmetro que pula de 5×50 para 50×200 é sinal de que não existe um ótimo estável.

5. Machine learning: o que é meta-labeling

A ideia mais comum, e mais perigosa, é pedir para um modelo **prever o preço** ("vai subir ou cair amanhã?"). Em forex isso é quase ruído puro, e o modelo acaba decorando o passado.

Aqui usamos uma técnica mais modesta e mais sólida, o **meta-labeling** (popularizada por Marcos López de Prado):

1. A **regra base** (cruzamento 20×50) continua decidindo **a direção** e **quando** entrar.
2. O modelo de ML só responde a uma pergunta de sim/não: **"este cruzamento específico vai dar lucro?"**
3. Se a probabilidade prevista for ≥ `PROB_MINIMA`, a operação é feita. Se não, ficamos **fora** até o próximo cruzamento.

Ou seja, o ML vira um **filtro**: ele não inventa operações, só pode pular as que parecem ruins. Como trend following perde em mercado lateral, a esperança é que o modelo aprenda a reconhecer "mercado lateral" pelas features e pule os *whipsaws*.

Rótulo (o que o modelo tenta prever)

Para cada operação da regra base: `1` se o resultado **líquido de custos** foi positivo, `0` se não.

Os dois modelos

Modelo	Por que usar
Regressão logística	simples, linear, difícil de sofrer overfitting, dá para ler os coeficientes
Random forest (rasa: profundidade 3)	capta relações não lineares (ex.: "ADX alto **e** volatilidade baixa"), mas limitada para não decorar

Os dois são leves: rodam em segundos no PC (nada de deep learning).

6. Features: as "pistas" que o modelo recebe

Todas são calculadas na **barra do sinal**, que já está fechada no momento da decisão (sem look-ahead). As que têm direção são multiplicadas pela direção da operação (+1 compra, −1 venda), para que "a favor da operação" seja sempre um número positivo, seja compra ou venda.

Feature	O que mede	Intuição
`adx`	força da tendência (ADX 14)	ADX baixo = mercado lateral, ruim para cruzamento

Feature	O que mede	Intuição
`atr_pct`	volatilidade (ATR 14 ÷ preço)	mercado muito parado mal paga os custos
`regime_vol`	ATR 14 ÷ ATR 100	a volatilidade está acelerando ou secando?
`incl_lenta`	inclinação da SMA 50 em 10 barras, em ATRs, a favor	tendência de fundo já apontando para o nosso lado?
`momento_20`	retorno das últimas 20 barras, em ATRs, a favor	o preço já andou a favor antes do cruzamento?
`dist_sma200`	distância do preço à SMA 200, em ATRs, a favor	a operação está alinhada com a tendência longa?
`cruzamentos_50`	nº de cruzamentos nas últimas 50 barras	muitos cruzamentos recentes = *whipsaw*
`direcao`	+1 compra, −1 venda	o par teve viés de alta ou de baixa?

```
584 operações com features completas | taxa de acerto da regra base: 38.0%
```

alvo	média nas perdedoras	média nas vencedoras
adx	21.5160	22.8560
atr_pct	0.2890	0.2960
regime_vol	0.9860	1.0230
incl_lenta	-0.1580	-0.0920
momento_20	2.8850	3.0910
dist_sma200	0.1470	0.8760
cruzamentos_50	1.9030	1.9050
direcao	0.0110	-0.0180

Primeira olhada: a tabela compara a média de cada feature nas operações vencedoras e perdedoras. Se as colunas forem quase iguais, as features **não separam** bons de maus cruzamentos e nenhum modelo vai fazer milagre. Diferenças claras (ex.: ADX maior nas vencedoras) são o que o modelo pode aproveitar.

> Atenção: esta tabela usa **todas** as operações, inclusive as do futuro. Ela serve só para entender os dados, **nunca** para escolher features. O teste honesto vem a seguir.

7. Treino walk-forward dos modelos

O mesmo princípio da seção 4, agora para o ML:

- Para testar o ano **A**, o modelo é treinado com as operações que **encerraram** antes de 1º de janeiro de **A**. Uma operação que abriu em dezembro e fechou em fevereiro **não entra** no treino de

fevereiro: em janeiro ainda não sabíamos o resultado dela (isso se chama *purging* e evita um vazamento sutil de informação).

- A janela de treino é **expansiva** (desde 2010): com só ~35 operações por ano, precisamos de toda a história disponível.
- O modelo prevê a probabilidade de acerto das operações que **abrem** no ano A. A operação é aceita se a probabilidade for ≥ `PROB_MINIMA`.
- Repetimos para todos os anos de 2014 em diante.

```
Operações fora da amostra: 461 | com previsão: 461
```

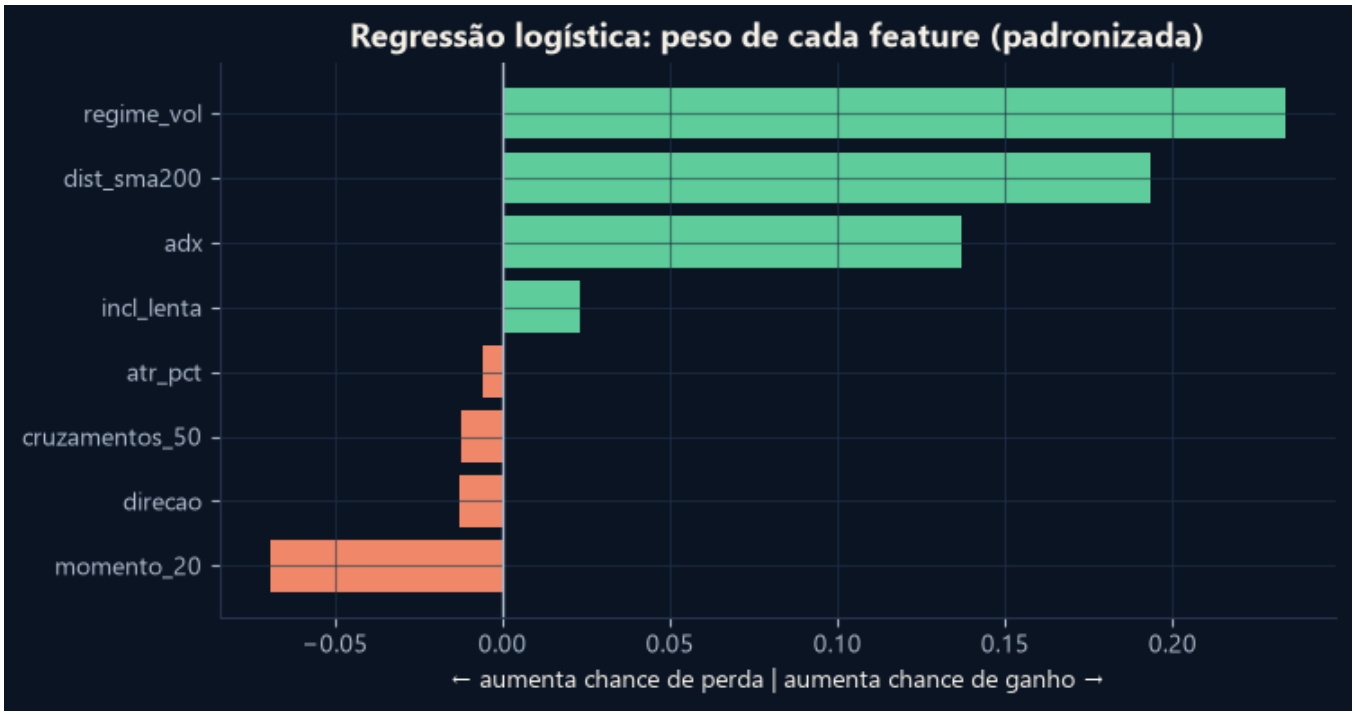
8. O filtro funcionou? Qualidade das previsões

Três maneiras de avaliar, da mais técnica à mais prática:

- **AUC**: probabilidade de o modelo dar nota maior a uma operação vencedora do que a uma perdedora sorteada. **0,50 = moeda**; acima de 0,55 já seria algo; acima de 0,60 é raro em mercado.
- **Acerto das aceitas × rejeitadas**: as operações que o modelo aceitou tiveram win rate maior que as que ele pulou?
- **Resultado médio**: o que importa no fim é a expectativa **em dinheiro** das aceitas ser maior que a das rejeitadas.

```
Win rate da regra base fora da amostra: 36.7%
```

	Logística	Random forest
AUC	0.5200	0.5080
% operações aceitas	11.4970	3.9050
Win rate aceitas %	52.8300	44.4440
Win rate rejeitadas %	34.5590	36.3430
Expectativa aceitas %	0.3020	0.0490
Expectativa rejeitadas %	-0.1080	-0.0650



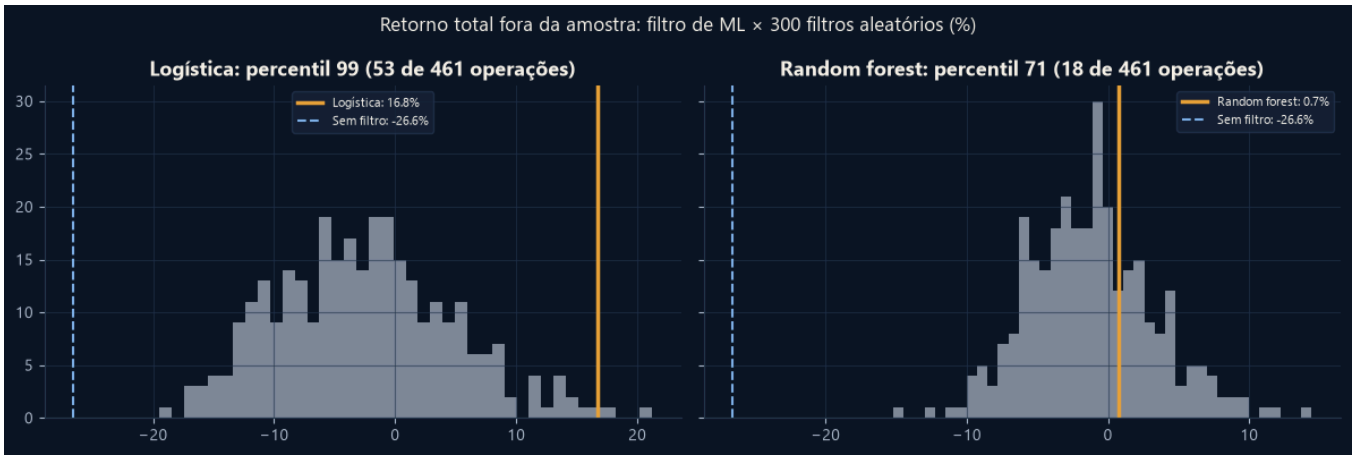
Como ler: barra verde = quanto maior a feature, maior a chance prevista de a operação dar lucro. Barra vermelha = o contrário. Pesos perto de zero = a feature quase não ajuda. Se os sinais fizerem sentido econômico (ex.: ADX e alinhamento com a SMA 200 positivos; muitos cruzamentos recentes negativo), é um bom sinal. Pesos "esquisitos" com AUC perto de 0,50 indicam que o modelo está lendo ruído.

9. Filtro de ML × filtros aleatórios

Pular operações **muda** o resultado, mesmo sem inteligência nenhuma. Então a pergunta certa é: *o filtro do modelo é melhor do que pular a **mesma quantidade** de operações ao acaso?*

Para cada modelo, sorteamos 300 filtros "macaco" que aceitam o mesmo nº de operações, escolhidas ao acaso entre as mesmas operações fora da amostra. Como a regra base está sempre posicionada, pular uma operação significa ficar fora nela, e o retorno total de qualquer filtro é simplesmente o produto dos resultados líquidos das operações aceitas.

Percentil = % dos filtros aleatórios que ficaram abaixo do modelo. Acima de 95% começa a ser interessante.



9b. Sensibilidade ao limiar de probabilidade

`PROB_MINIMA = 0,50` foi definido **antes** de ver os resultados, mas um bom filtro não pode depender de um número mágico (o mesmo raciocínio do mapa de calor do 001). Recalculamos o filtro com vários limiares. **Isto não serve para escolher o melhor limiar** (seria otimizar olhando o teste), só para ver se a vantagem aparece numa faixa ampla ou só num ponto.

	operações aceitas		retorno total %		percentil vs aleatório	
modelo	**Logística**	**Random forest**	**Logística**	**Random forest**	**Logística**	**Random forest**
limiar						
0.3500	309	346	-6.4000	-27.2000	87.7000	22.0000
0.4000	187	170	0.3000	-5.8000	84.7000	63.3000
0.4200	147	116	4.0000	1.4000	87.7000	83.0000
0.4500	98	75	8.7000	13.7000	91.0000	95.3000
0.4800	66	40	14.7000	13.7000	98.7000	97.7000
0.5000	53	18	16.8000	0.7000	99.7000	67.3000
0.5500	28	3	1.7000	-1.1000	72.3000	32.0000

10. Comparação final: fora da amostra (2014 → hoje)

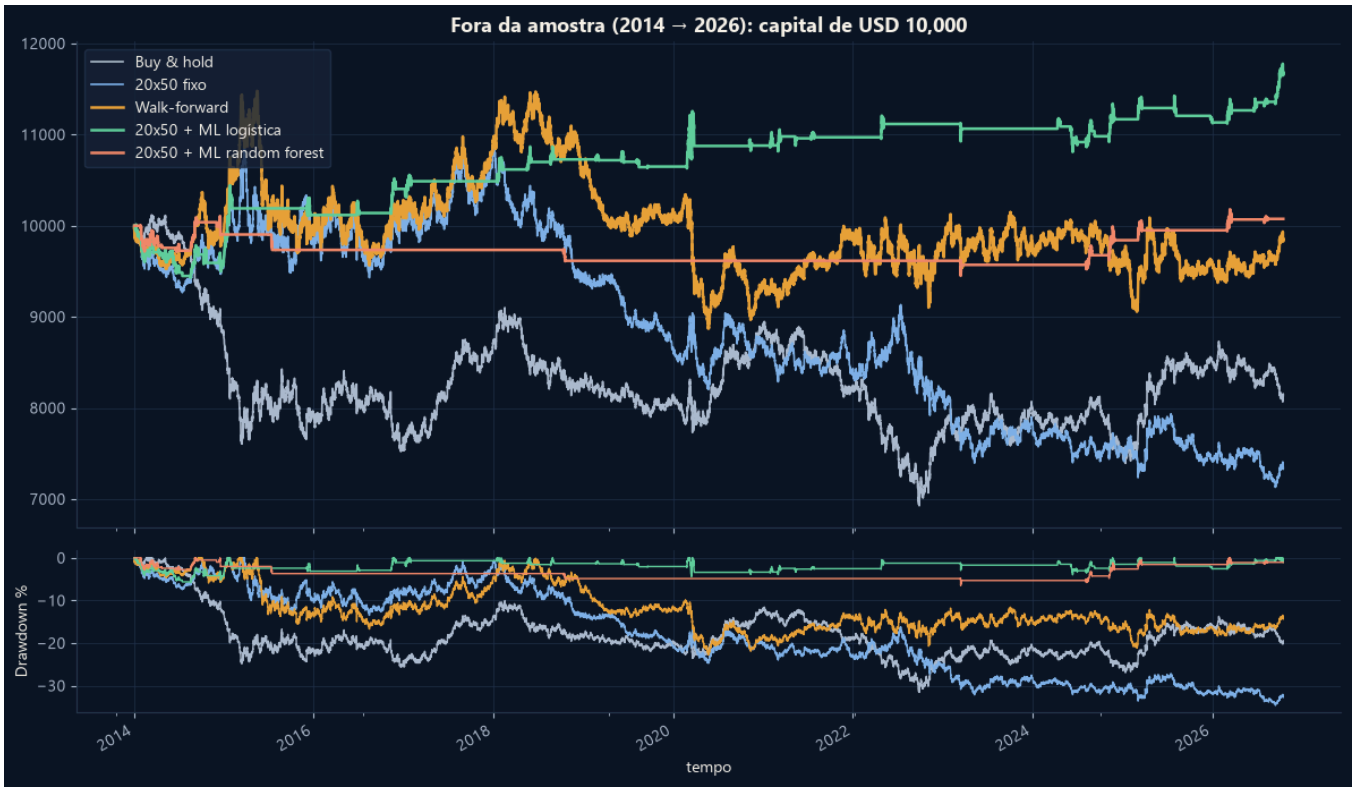
Todas as versões medidas no **mesmo período** fora da amostra, com **os mesmos custos**:

Versão	O que é
Buy & hold	ficar comprado o tempo todo
20×50 fixo	a regra do 001, sem nada a mais
Walk-forward	médias reescolhidas todo ano (seção 4)

Versão	O que é
20×50 + ML logística / random forest	regra base, pulando as operações que o modelo rejeitou (seções 7–9)

As versões com filtro são montadas zerando a posição nas operações rejeitadas e recalculando tudo pelo mesmo motor `retornos()`.

	Buy & hold	20x50 fixo	Walk-forward	20x50 + ML logística	20x50 + ML random forest
Retorno total %	-18.6300	-26.5700	-1.5100	16.7500	0.7500
CAGR %	-1.6000	-2.3900	-0.1200	1.2200	0.0600
Nº trades	NaN	461.0000	228.0000	53.0000	18.0000
Win rate %	NaN	36.6600	35.0900	52.8300	44.4400
Payoff	NaN	1.4900	1.8800	1.8300	1.3800
Expectativa %/trade	NaN	-0.0600	0.0100	0.3000	0.0500
Fator de lucro	NaN	0.8600	1.0200	2.0500	1.1100
Máx. drawdown %	-31.5500	-34.5700	-22.7100	-5.7900	-6.4900
Sharpe	-0.1700	-0.2700	0.0200	0.4200	0.0500
Sortino	-0.2300	-0.3800	0.0300	0.6100	0.0600



11. Salvar os resultados

Pasta `backtests/001.1_sma_wf_ml_EURUSD_H4/` :

- `parametros.json` : configuração exata;
- `metricas.csv` : comparação final fora da amostra;
- `walkforward.csv` : combinação escolhida em cada ano, com Sharpe de treino e de teste;
- `ml_operacoes_oos.csv` : cada operação fora da amostra com as features, a probabilidade de cada modelo e se foi aceita;
- `ml_qualidade.csv` : AUC e acerto das aceitas × rejeitadas.

```
Resultados salvos em C:\Dev\GitHub\LHS Capital\lhs-quant-lab\backtests\001.1_sma_wf_ml_EURUSD
_H4
C:\Users\luizh\AppData\Local\Temp\ipykernel_11412\2232736183.py:19: UserWarning: obj.round ha
s no effect with datetime, timedelta, or period dtypes. Use obj.dt.round(...) instead.
  ds_oos.round(5).to_csv(PASTA_BACKTEST / "ml_operacoes_oos.csv", index=False, encoding="utf-
8")
```

12. Conclusões e exercícios

O que encontramos (rodada de 18/10/2026 — EURUSD H4, fora da amostra 2014 → out/2026)

Versão	Retorno	Sharpe	Máx. DD	Trades	Fator de lucro
Buy & hold	−18,6%	−0,17	−31,6%	—	—
20×50 fixo (regra do 001)	−26,6%	−0,27	−34,6%	461	0,86
Walk-forward	−1,5%	0,02	−22,7%	228	1,02
20×50 + ML logística	**+16,8%**	**0,42**	**−5,8%**	**53**	2,05
20×50 + ML random forest	+0,8%	0,05	−6,5%	18	1,11

Walk-forward: melhorou o resultado (de −26,6% para −1,5%) e bateu a 20×50 fixa em 9 de 13 anos, mas **não criou vantagem**: Sharpe médio de 0,30 no treino caiu para 0,04 no teste (**WFE = 0,14**). O "melhor parâmetro" de cada janela era em grande parte sorte. Uma pista útil: a otimização escolheu médias lentas de **150–200** em 10 de 13 anos.

Machine learning (meta-labeling):

- A **logística** foi o único resultado promissor: as operações aceitas tiveram win rate de **53%** contra 35% das rejeitadas, e o filtro ficou no **percentil 99** contra filtros aleatórios. A vantagem se manteve (percentil 85–99) em toda a faixa de limiares de 0,35 a 0,50.
- **Mas:** o AUC foi só **0,52** (quase moeda); o filtro aceitou só **53 operações em 13 anos** (20 delas em 2014) e a random forest, com as mesmas features, **não confirmou** (percentil 71). Com tão poucos

trades, o resultado ainda pode ser sorte.
- A feature que mais separa vencedoras de perdedoras é `dist_sma200` (operar **a favor da SMA 200**), coerente com a preferência do walk-forward por médias de 200. A vantagem talvez venha de uma regra simples, sem ML nenhum.

Veredito: 🟡 promissor, ainda não aprovado. Nenhuma versão vai para conta demo ou real antes de passar no próximo teste. Isso é o rigor funcionando: o resultado bonito (+16,8%, DD de −5,8%) chama atenção, mas tem poucos trades e só um dos dois modelos o confirma.

> Os números acima mudam conforme os dados forem atualizados. Ao rodar de novo, confira as tabelas.

Próximo passo sugerido (Backtest 001.2 ou 002)

Testar a regra **sem ML**: *20×50 só a favor da SMA 200* (só compra acima, só vende abaixo). Se ela reproduzir a melhora, a vantagem é simples e explicável. Depois disso, validar em **GBPUSD, USDJPY e D1** antes de qualquer conta demo.

Exercícios

1. Mude `SIMBOLO` para `"GBPUSD"` e rode tudo. O filtro da logística continua no percentil > 95? (É o teste mais importante.)
2. Remova `dist_sma200` de `FEATURES`. O filtro ainda funciona sem ela?
3. Troque `ANOS_TREINO` para 3 e 6. O WFE melhora?
4. Coloque `SLIPPAGE_PONTOS = 10`. Quais versões sobrevivem a uma execução ruim?
5. Aumente `max_depth` da random forest para 8. O resultado fora da amostra melhora ou piora? (Dica: overfitting.)